

Final Project Report

ExpenseTracker

A Full-Stack MERN Personal Expense & Budget Management System

Project Title	ExpenseTracker — Personal Expense & Budget Management System
Submitted By	Sharma Saurabh
Roll Number	250863131007
Institute	Laxmi Institute of Technology (LIT), Sarigam — Laxmi Vidyapeeth
Department	Computer Science & Engineering
University	Gujarat Technological University (GTU)
Project Guide	X
Academic Year	2025–2026
Report Date	08 October 2026

Abstract

Managing personal money with notebooks or scattered spreadsheets is slow, error-prone and gives no real picture of spending habits. ExpenseTracker is a web application built on the MERN stack (MongoDB, Express.js, React.js, Node.js) that lets a user record income and expenses, organise them into categories, set monthly budgets, track savings goals, automate recurring payments and understand spending through interactive charts and monthly reports.

The system exposes a REST API secured with JWT authentication, uses a Redux Toolkit powered React front end, and stores data in MongoDB. Beyond basic CRUD it provides smart de-duplicated notifications, month-over-month reporting with percentage change, CSV export, receipt uploads, tags, favorites, light/dark mode and eight selectable colour themes. All data is strictly per-user and every amount is validated on both the client and the server. The project runs entirely locally with two commands and depends on no third-party services.

Table of Contents

1. Introduction
2. Objectives
3. Technology Stack
4. System Design
5. Database Design
6. Modules & Features
7. API Reference
8. User Interface
9. Implementation Highlights
10. Security & Validation
11. Testing
12. Challenges & Solutions
13. Limitations
14. Future Scope
15. Conclusion
16. How to Run
17. Appendix A — Environment Variables

18. Appendix B — Seed Data

1. Introduction

1.1 Background

Most people know roughly what they earn but not where it goes. Tracking every expense by hand is tedious, and existing apps are often cluttered, ad-driven or locked behind subscriptions. A simple, private, self-hosted tracker solves this for students and individuals.

1.2 Problem Statement

There is a need for a lightweight web application that lets a user record transactions quickly, stay within monthly budgets, work towards savings goals, and see clear analytics — without relying on third-party services or internet connectivity beyond the local machine.

1.3 Proposed Solution

ExpenseTracker provides a single dashboard for all of this. Users register, add categories and transactions, set per-category monthly budgets, create savings goals and recurring expenses, and view analytics and monthly reports. Recurring items post themselves when due, notifications warn before budgets break, and every screen offers search, filters and export.

1.4 Scope

In scope: authentication, transaction management, categories, budgets, savings goals, recurring expenses, analytics, monthly reports, notifications, CSV export, receipt upload and appearance customisation (themes + dark mode). Out of scope: bank integration, multi-user shared accounts and native mobile apps — these are listed under Future Scope.

2. Objectives

1. Build a full-stack application covering the complete finance workflow: transactions → budgets → goals → reports.
2. Provide secure user accounts where each user can only access their own data.
3. Give meaningful insight through interactive charts and month-over-month comparison.
4. Warn users automatically when budgets are nearly or fully used, without repeating the same warning.
5. Offer a clean, consistent and accessible interface with light/dark mode and selectable themes.
6. Validate every amount on both ends so bad data can never reach the database.

7. Keep the code simple and modular: small single-purpose files and sequential `async/await` flow that is easy to explain and maintain.

3. Technology Stack

Layer	Technology	Purpose
Database	MongoDB (local, DB <code>expenseTracker</code>)	Flexible document storage for finance records
Backend	Node.js, Express.js	REST API on port 5000
ODM	Mongoose 8	Schemas, validation, references, aggregation
Frontend	React 18 (Vite)	Component-based SPA on port 5173
State	Redux Toolkit	Predictable state via eight slices
Routing	React Router v6	Protected and public routes
Charts	Recharts	Bar, line, area, pie and donut charts
Styling	Tailwind CSS + CSS variables	Consistent styling, theming, dark mode
HTTP	Axios	Central instance with auth/error interceptors
Auth	JWT, bcryptjs	Stateless auth, hashed passwords
Uploads	Multer	Receipt images (max 2 MB, image types only)
Security	helmet, CORS allow-list, morgan	Secure headers, restricted origins, request log
Feedback	react-hot-toast	Success/error toasts

The Vite dev server proxies `/api` and `/uploads` to the backend, so the browser only ever talks to one origin and CORS stays trivial in development.

4. System Design

4.1 Architecture

ExpenseTracker follows a three-tier client–server architecture: a React single-page application (presentation), an Express REST API (application) and MongoDB (data). No page ever talks to the database directly — every read or write passes through the API, which authenticates the token and scopes the query to that user.

Data flow: Page → Redux slice → `services/api.js` (Axios + JWT header) → Express route → middleware (`protect`) → controller → Mongoose model → MongoDB → JSON response → slice state → UI and charts.

4.2 Request Lifecycle

1. Client sends `Authorization: Bearer <token>` (attached automatically by the Axios request interceptor).
2. `protect` middleware verifies the JWT and loads `req.user`.
3. Controller validates/sanitises input (for example `toAmount`), then runs the query with `user: req.user._id`.
4. Response returns a standardised `{ success, data/message }` envelope.
5. On HTTP 401 the Axios response interceptor clears the token and redirects to `/login`.
6. Any thrown error lands in the central error handler, which maps `CastError` → 404, duplicate key → 400, `Mongoose ValidationError` → 400.

4.3 Project Structure

```
ExpenseTracker/
├── server/
│   └── src/
│       ├── config/env.js      # only file reading process.env
│       ├── config/db.js      # mongoose connection
│       ├── models/           # 7 Mongoose models
│       ├── middleware/       # auth (JWT), error, upload (Multer)
│       ├── utils/            # notify, processRecurring, budgetStatus,
│                               # buildCsv, sanitize, defaultCategories
│       ├── controllers/      # 8 controllers
│       ├── routes/           # 8 routers (40 endpoints)
│       ├── seed/seed.js      # 2 demo users + data
│       └── app.js, server.js
└── client/
    └── src/
        ├── config/env.js      # only file reading import.meta.env
        ├── themes/            # 8 theme files + registry
        ├── features/          # 8 Redux slices + store
        ├── services/api.js    # Axios instance + interceptors
        ├── components/        # Modal, EmptyState, Loader, ProgressBar,
        │                       # charts, Sidebar/Navbar/MainLayout
        ├── pages/             # 11 pages
        ├── hooks/
        ├── utils/
        └── index.css          # CSS variables + component classes
```

5. Database Design

Collection	Key fields
User	name, email (unique index), password (bcrypt hash), currency, monthlyIncome, theme preferences
Category	user, name, type (income/expense), icon, color
Transaction	user, type, amount, category (ref), description, date, paymentMethod, tags[], receipt, favorite, notes, isRecurring, recurringExpense
Budget	user, category (ref), amount, month, year — unique per user+category+month
SavingsGoal	user, title, targetAmount, currentAmount, targetDate, icon
RecurringExpense	user, title, amount, category, frequency, nextDate, paymentMethod, active

Notification	user, type, title, message, key (de-duplication), read
---------------------	--

Finance collections are indexed on `user` together with date or month/year for fast aggregation. Every query filters by the logged-in user's id, so users can never see each other's data. Categories are referenced (not copied), so renaming a category updates every view of it.

6. Modules & Features

#	Module	Description
1	Authentication	Register, login, fetch current user, update profile (name, currency, monthly income target); JWT stored in localStorage
2	Dashboard	Total balance hero card, month income/expense, six-month bar chart, category donut, budget status bars, recent transactions
3	Transactions	Add/edit/delete income and expenses; debounced search, pagination; filters by date range, category, type, payment method, amount range, tags and favorites-only; row flash on insert
4	Categories	Custom categories with icon and colour; separate income and expense types; deletion blocked while a recurring expense still uses the category
5	Budgets	Per-category monthly limits with progress bars and on-track / near-limit ($\geq 80\%$) / over-budget ($\geq 100\%$) status
6	Savings Goals	Target vs current amount, add-money contributions, progress percentage, milestone notification at 100%
7	Recurring Expenses	Monthly payments with active toggle; due items are posted automatically as real transactions with <code>isRecurring</code> flag and next date advanced
8	Analytics	Income vs expense by month, spending by category, payment-method split; vertical selector switches Bar, Line, Area, Donut and Pie instantly
9	Monthly Reports	Month navigator, month-over-month insight banner ("spent 48% less than last month"), six-month chart, category table with change %, top categories, payment-method breakdown
10	Notifications	Budget warning ($\geq 80\%$), budget exceeded ($\geq 100\%$), goal reached, recurring-due and welcome alerts — de-duplicated by a stable key in a bell dropdown
11	CSV Export	Downloads the currently filtered transaction list as a date-stamped CSV file
12	Receipts & Favorites	Attach an image (≤ 2 MB) to a transaction with lightbox preview; star frequent transactions and open a favorites-only list
13	Appearance	Light/dark mode toggle and eight selectable colour themes (Option 1–8), both persisted in localStorage

14	UX Polish	Loading skeletons on every page, friendly empty states, 150–250 ms CSS micro-interactions, Escape-to-close modals, visible focus outlines
15	Validation	Amount sanitization while typing (digits, one dot, max two decimals) mirrored by server-side <code>toAmount</code> checks

7. API Reference (40 endpoints + health check)

Router	Endpoints
<code>/api/auth</code>	POST <code>/register</code> , POST <code>/login</code> , GET <code>/me</code> , PUT <code>/profile</code>
<code>/api/transactions</code>	GET <code>/</code> , POST <code>/</code> , GET <code>/:id</code> , PUT <code>/:id</code> , DELETE <code>/:id</code> , GET <code>/csv</code> , GET <code>/favorites</code> , PATCH <code>/:id/favorite</code> , PATCH <code>/:id/receipt</code>
<code>/api/categories</code>	GET <code>/</code> , POST <code>/</code> , PUT <code>/:id</code> , DELETE <code>/:id</code>
<code>/api/budgets</code>	GET <code>/</code> , POST <code>/</code> , PUT <code>/:id</code> , DELETE <code>/:id</code>
<code>/api/goals</code>	GET <code>/</code> , POST <code>/</code> , PUT <code>/:id</code> , DELETE <code>/:id</code> , POST <code>/:id/add-money</code>
<code>/api/recurring-expenses</code>	GET <code>/</code> , POST <code>/</code> , PUT <code>/:id</code> , DELETE <code>/:id</code> , PATCH <code>/:id/toggle</code>
<code>/api/analytics</code>	GET <code>/dashboard</code> , <code>/monthly</code> , <code>/categories</code> , <code>/payment-methods</code> , <code>/reports</code>
<code>/api/notifications</code>	GET <code>/</code> , PATCH <code>/read-all</code> , PATCH <code>/:id/read</code> , DELETE <code>/:id</code>
<code>/api/health</code>	GET <code>/</code> — liveness check used during testing

Every route except register, login and health sits behind the `protect` middleware. All responses share the same `{ success, data | message }` envelope, which keeps the front-end error handling uniform.

8. User Interface

8.1 Design System — Option 8 (Warm White + Espresso + Sage)

The default theme uses a warm white background with espresso brown and sage green accents. All colours are CSS variables (`--bg` , `--text` , `--card` , `--border` , `--primary` , `--accent` , `--sage` , `--beige` , ...) defined for both light and dark mode, so changing theme or mode only swaps variables — no component ever contains colour logic. The full interface uses the JetBrains Mono typeface for a distinctive engineering-first look.

Token	Hex	Usage
Background	#FAF8F3	Page background
Text	#29231F	Primary text
Primary (Espresso)	#493B32	Buttons, active states, sidebar brand
Accent (Sage)	#78816A	Charts, links, highlights
Sage tint	#E9EDE4	Sidebar, soft panels
Beige tint	#E9DFD1	Balance and stat cards
Card	#FFFFFF	Surfaces in light mode

8.2 Theme Engine

Each of the eight themes lives in its own file under `client/src/themes/` (Warm Ivory Forest, Off-White Navy, Cream Burgundy, White Terracotta, White Olive, Pearl Plum, White Cobalt, Warm Espresso Sage), with a small registry exposing `getTheme()` and `applyTheme()` . The choice is made under Profile → Appearance and persisted in `localStorage` along with the dark-mode flag; `App.jsx` re-applies both on every load. Dark mode is an independent toggle available in the navbar.

8.3 Interaction & Accessibility

Transitions (150–250 ms) are pure CSS: hover lift, modal fade+scale, progress-bar fill, skeleton shimmer and a flash highlight on newly added rows. Modals close with Escape, inputs are labelled, and focus outlines are visible for keyboard users. Amount inputs use `inputMode="decimal"` and hide the browser number spinners. Every page has a loading skeleton and a friendly empty state instead of a blank table.

8.4 Screenshots

Insert your own screenshots in the boxes below (replace each box with an tag).

Screenshot: Login page

Figure 1 — Login

Screenshot: Dashboard

Figure 2 — Dashboard with balance, charts and budget status

Screenshot: Transactions list and Add Transaction modal

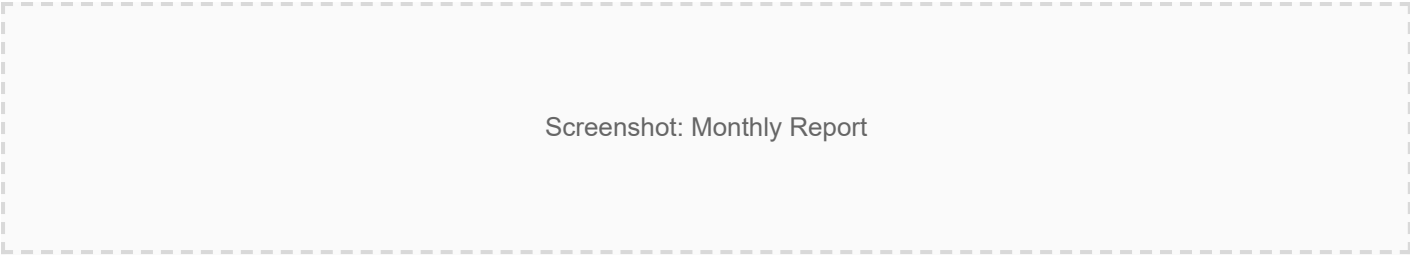
Figure 3 — Transactions with filters and CSV export

Screenshot: Budgets

Figure 4 — Monthly budgets

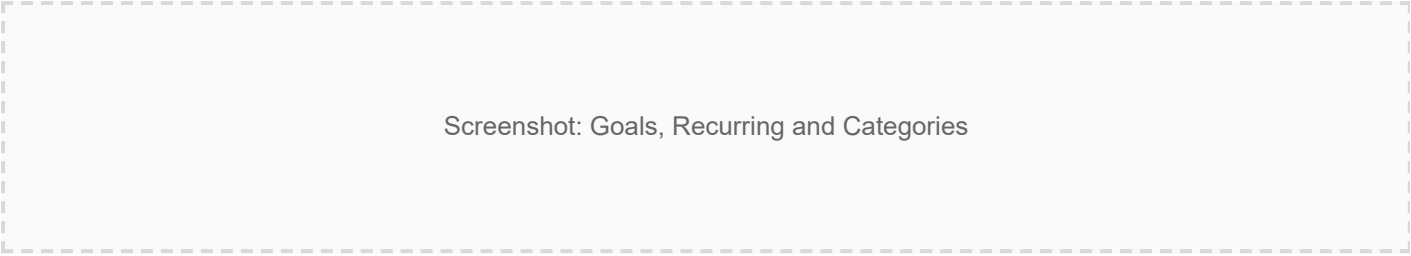
Screenshot: Analytics (Bar and Line views)

Figure 5 — Analytics with chart-type selector



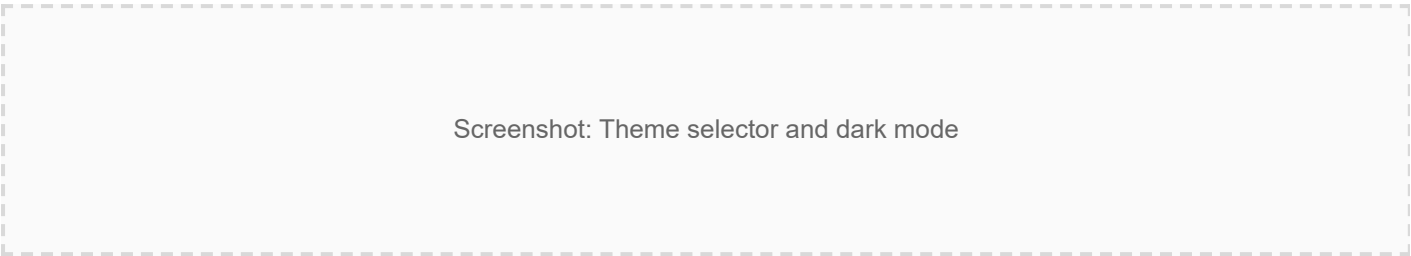
Screenshot: Monthly Report

Figure 6 — Monthly report with month-over-month insight



Screenshot: Goals, Recurring and Categories

Figure 7 — Savings goals, recurring expenses and categories



Screenshot: Theme selector and dark mode

Figure 8 — Appearance settings and dark mode

9. Implementation Highlights

9.1 Recurring Auto-Posting

When the app loads, every active recurring item whose `nextDate` is today or earlier is converted into a real transaction, its next due date is advanced by one frequency step, and a notification is created. The posted transaction keeps a link back to its recurring parent so the UI can mark it.

```
const dueItems = await RecurringExpense.find({
  user: userId, active: true, nextDate: { $lte: today },
});
for (const item of dueItems) {
  await Transaction.create({ user: userId, type: 'expense',
    amount: item.amount, category: item.category,
    description: item.title, date: item.nextDate,
    paymentMethod: item.paymentMethod, isRecurring: true,
    recurringExpense: item._id });
  item.nextDate = nextDateAfter(item.nextDate, item.frequency);
  await item.save();
}
```

9.2 Smart Notifications with De-duplication

Each alert carries a stable key such as `budget:2026:10:Food:warn`. Before creating a notification the server checks whether that key already exists, so the same warning never repeats inside a month — the bell stays useful instead of noisy.

```
const createOnce = async (userId, type, title, message, key) => {
  const alreadySent = await Notification.findOne({ user: userId, key });
  if (alreadySent) return;
  await Notification.create({ user: userId, type, title, message, key });
};
```

The budget checker calls it at $\geq 80\%$ ("used 84%") and $\geq 100\%$ ("over by 12%"), goals fire once when fully funded, and recurring items confirm each posting.

9.3 Budget Status Calculation

For each month the server loads the user's budgets, then aggregates that category's expenses inside the same month window (`$match` on user, type, category and date range, then `$group` with `$sum`).

Percentage, remaining amount and status are derived and returned together, so the dashboard, budgets page and reports all read one source of truth.

9.4 Dashboard & Report Aggregation

Totals, six-month series and category breakdowns are computed with MongoDB aggregation pipelines executed with sequential `await` calls for readability. A notable bug was fixed here: inside `$group`, non-`_id` fields must be accumulated — bare paths raise *"The field must be an accumulator object"* — so fields like name and type use `{ $first: '$field' }`.

9.5 Amount Validation (client + server)

```
const toAmount = (value) => {
  const number = Number(value);
  if (!Number.isFinite(number) || number <= 0) return null;
  return Math.round(number * 100) / 100; // 123.456 -> 123.46
};
```

The client's `sanitizeAmountInput` strips letters and minus signs while typing and caps the decimal places, and the server independently converts and rejects invalid values with HTTP 400 — so `abc`, `-5` or a zero budget never reach the database.

9.6 Chart Selector

Analytics keeps the fetched data mounted and only swaps the chart component (Bar / Line / Area / Donut / Pie), so switching views is instant and never re-fetches.

9.7 CSV Export

`GET /api/transactions/csv` applies the same filters as the on-screen list and streams a `text/csv` response with a date-stamped filename, generated by a small `buildCsv` helper.

9.8 Axios Interceptors

One request interceptor attaches `Bearer` tokens from `localStorage`; one response interceptor reacts to 401 by clearing the token and bouncing the user to `/login`. There is exactly one HTTP client in the app, so auth behaviour is defined in one place.

10. Security & Validation

1. Passwords are hashed with bcrypt (10 rounds) and never returned by the API.

2. JWT with configurable expiry is sent as an `Authorization: Bearer` header; the Axios interceptor attaches it and clears it on failure.
3. Ownership checks: every find, update and delete includes the logged-in user's id.
4. Server-side validation through `sanitize.js` returns 400 with clear messages for invalid amounts, empty titles and bad enum values.
5. helmet security headers and a CORS allow-list restricted to the client URL; morgan logs every request in development.
6. Multer restricts receipt uploads to image MIME types up to 2 MB.
7. A central error handler maps `CastError` → 404, `duplicate key` → 400, `ValidationError` → 400 and never leaks stack traces to the client.
8. All environment variables are read in exactly two files (`server/src/config/env.js` , `client/src/config/env.js`); secrets stay in a git-ignored `.env` with a committed `.env.example` .

11. Testing

Area	Test	Result
Health	GET /api/health	Pass (200)
Auth	Register, login, get current user, update profile	Pass (200)
Dashboard	Summary aggregation (after accumulator fix)	Pass (200)
Reports	Month-over-month, category change %, payment methods	Pass (200)
CRUD	Transactions, budgets, goals, recurring, categories	Pass (200)
CSV	Export endpoint	Valid CSV stream
Validation	Amount abc , -5 ; budget 0	Rejected with 400
Validation	Amount 123.456	Stored as 123.46
Categories	Delete category used by a recurring expense	Blocked with clear message
Frontend	npm run build	Success (948 modules)
Seeding	Demo and Saurabh users with data	Verified (31 / 18 transactions)
Visual	Every page rendered and captured (desktop, light + dark)	Verified

12. Challenges & Solutions

Challenge	Solution
MongoDB error “field must be an accumulator object” in \$group	Used accumulators such as { \$first: '\$name' } for non- _id fields in three pipelines
Custom .table-row class clashed with Tailwind's generated display: table-row utility and broke a flex row	Renamed the component class to .row-hover ; built CSS now contains zero table-row rules
Amount fields accepted letters, negatives and extra decimals	Added sanitizeAmountInput on the client and toAmount on the server, tested with API calls
Port conflicts (EADDRINUSE) on restart	One dev process per port, ports defined in .env

Environment variables scattered across files	Restricted raw <code>process.env</code> / <code>import.meta.env</code> access to exactly two config files
Same budget warning repeating every refresh	Stable notification keys with a pre-insert existence check (de-duplication)

13. Limitations

- Recurring expenses are posted only when the app is opened, not by a background scheduler.
- The production build shows a chunk-size warning (>500 kB) because Recharts bundles all chart types in one bundle.
- Only monthly frequency and a single currency per user are supported; no live exchange rates.
- No automated test suite yet; testing was done manually and with API calls.
- Receipt images are stored on local disk, not in the database or cloud storage.

14. Future Scope

1. Cron scheduler (node-cron) to post recurring expenses without opening the app.
2. Monthly report emails and budget-breach push/email alerts.
3. Bank statement (CSV/OFX) import and receipt OCR for automatic entry.
4. Smart features: automatic category suggestion, anomaly detection and month-end forecasting.
5. Multi-currency support with daily exchange rates and shared/family budgets.
6. PWA with offline mode and a React Native mobile app using the same REST API.
7. Rate limiting, refresh-token rotation and an audit log of edits/deletes.
8. Jest + Supertest API tests, React Testing Library component tests, CI/CD pipeline.
9. Route-level code splitting to remove the bundle-size warning.
10. Gamification (streaks, savings badges) and chart image export for presentations.

15. Conclusion

ExpenseTracker delivers an end-to-end MERN application that covers the full personal-finance workflow — transactions, budgets, savings goals, recurring payments, analytics and reports — with secure per-user data, validation on both client and server, and a themeable, accessible interface built on a distinctive design system. The modular structure (small files, sequential async flow, centralised configuration and a

single HTTP client) keeps the project easy to understand, maintain and demonstrate. The testing table and fixed-challenge log show a working, verified system, while the future-scope items provide a clear path for growth beyond the current semester's requirements.

16. How to Run

Prerequisites: Node.js 18 or later and MongoDB running locally on port 27017.

```
# Backend
cd server
npm install
cp .env.example .env
npm run seed      # creates demo + saurabh users with data
npm run dev       # http://localhost:5000

# Frontend (new terminal)
cd client
npm install
npm run dev       # http://localhost:5173
```

Demo logins:

Email	Password	Data
demo@example.com	password123	31 transactions, 5 budgets, 5 recurring, 3 goals
saurabh@gmail.com	saurabh123	18 transactions, 3 budgets, 3 recurring, 3 goals

Appendix A — Environment Variables

server/.env

```
PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/expenseTracker
JWT_SECRET=<your-secret>
JWT_EXPIRES_IN=7d
CLIENT_URL=http://localhost:5173
```

client/.env

```
VITE_API_BASE_URL=/api
VITE_APP_NAME=ExpenseTracker
VITE_APP_TAGLINE=Track every rupee
VITE_REQUEST_TIMEOUT=15000
```

Appendix B — Seed Data

`npm run seed` wipes the database and recreates two ready-to-demo accounts (September + October 2026 data so charts, reports and month-over-month comparisons all have content):

User	Email	Password	Contents
Demo User	demo@example.com	password123	31 transactions, 5 budgets, 5 recurring expenses, 3 goals
Saurabh	saurabh@gmail.com	saurabh123	18 transactions, 3 budgets, 3 recurring expenses, 3 goals

Both accounts receive the 16 default categories (8 income + 8 expense), and each recurring plan includes one item already due so the auto-posting feature demonstrates itself on first login.